



Data Engineering

MINI PROJECT



Sales Data Pipeline



Mini Project: Sales Data ETL Pipeline Using PostgreSQL & Python

Project Title

Building a Data Engineering Pipeline with PostgreSQL and Pandas

Project Objective

Students will learn how to:

- ✓ Connect to a PostgreSQL database
- ✓ Extract data using SQL queries
- ✓ Clean and transform data using Pandas
- ✓ Perform data quality checks
- ✓ Generate business insights
- ✓ Export processed data to CSV and JSON
- ✓ Build a complete ETL pipeline

Business Scenario

A retail company stores all sales transactions in a PostgreSQL database. Management wants a clean and analyzed dataset for reporting and dashboard creation.

As a Data Engineer, your task is to:

1. Extract sales data from PostgreSQL.
2. Clean and validate the data.
3. Create calculated metrics.
4. Generate reports.
5. Export processed data.

🔧 Technologies Used

- Python
 - PostgreSQL
 - Pandas
 - SQLAlchemy
 - psycopg2
 - Jupyter Notebook
-

📦 Installation

```
pip install pandas sqlalchemy psycopg2-binary openpyxl
```

🗄 Database Schema

Table: customers

```
CREATE TABLE customers (  
    customer_id SERIAL PRIMARY KEY,  
    customer_name VARCHAR(100),  
    city VARCHAR(100),  
    registration_date DATE  
);
```

Table: products

```
CREATE TABLE products (  
    product_id SERIAL PRIMARY KEY,  
    product_name VARCHAR(100),  
    category VARCHAR(50),  
    price NUMERIC(10,2)  
);
```

Table: sales

```
CREATE TABLE sales (  
  sale_id SERIAL PRIMARY KEY,  
  customer_id INT,  
  product_id INT,  
  quantity INT,  
  sale_date DATE,  
  FOREIGN KEY(customer_id) REFERENCES customers(customer_id),  
  FOREIGN KEY(product_id) REFERENCES products(product_id)  
);
```

Sample Dataset

Customers

customer_id	customer_name	city
-------------	---------------	------

1	Abebe	Addis Ababa
2	Hana	Adama
3	Samuel	Hawassa

Products

product_id	product_name	category	price
------------	--------------	----------	-------

1	Laptop	Electronics	800
2	Mouse	Electronics	15
3	Chair	Furniture	120

Sales

sale_id	customer_id	product_id	quantity
1	1	1	2
2	2	2	5
3	3	3	1

ETL Pipeline

Step 1: Connect to PostgreSQL

```
from sqlalchemy import create_engine
```

```
engine = create_engine(  
    "postgresql://postgres:password@localhost:5432/retail_db"  
)
```

Step 2: Extract Data

```
import pandas as pd
```

```
query = """  
SELECT  
    s.sale_id,  
    s.sale_date,  
    c.customer_name,  
    c.city,  
    p.product_name,  
    p.category,  
    p.price,  
    s.quantity  
FROM sales s  
JOIN customers c  
    ON s.customer_id = c.customer_id
```

```
JOIN products p
  ON s.product_id = p.product_id
"""
```

```
df = pd.read_sql(query, engine)
```

```
df.head()
```

Data Exploration

Dataset Shape

```
df.shape
```

Data Types

```
df.dtypes
```

Summary Statistics

```
df.describe()
```

Check Null Values

```
df.isnull().sum()
```

Data Cleaning

Remove Duplicates

```
df.drop_duplicates(inplace=True)
```

Handle Missing Values

```
df["quantity"] = df["quantity"].fillna(1)
```

```
df["city"] = df["city"].fillna("Unknown")
```

Standardize Text

```
df["customer_name"] = df["customer_name"].str.title()
```

```
df["city"] = df["city"].str.title()
```

Remove Negative Quantities

```
df = df[df["quantity"] > 0]
```

⚙ Data Transformation

Calculate Total Sale

```
df["total_sale"] = (  
    df["price"] * df["quantity"]  
)
```

Convert Date Column

```
df["sale_date"] = pd.to_datetime(df["sale_date"])
```

Extract Date Features

```
df["year"] = df["sale_date"].dt.year  
df["month"] = df["sale_date"].dt.month  
df["month_name"] = df["sale_date"].dt.month_name()  
df["day"] = df["sale_date"].dt.day
```

Create Sales Category

```
def sales_category(amount):
```

```
    if amount >= 1000:  
        return "High"
```

```
    elif amount >= 500:  
        return "Medium"
```

```
    return "Low"
```

```
df["sales_category"] = (  
    df["total_sale"]  
    .apply(sales_category)  
)
```

Business Analysis

Top Selling Products

```
top_products = (  
    df.groupby("product_name")  
        ["quantity"]  
        .sum()  
        .sort_values(ascending=False)  
)
```

```
top_products
```

Revenue by Category

```
revenue_category = (  
    df.groupby("category")  
        ["total_sale"]  
        .sum()
```



```
    .sort_values(ascending=False)
)
```

revenue_category

Revenue by City

```
city_sales = (
    df.groupby("city")
      ["total_sale"]
      .sum()
      .sort_values(ascending=False)
)
```

city_sales

Top Customers

```
top_customers = (
    df.groupby("customer_name")
      ["total_sale"]
      .sum()
      .sort_values(ascending=False)
)
```

top_customers.head(10)

Pivot Table

```
pivot = pd.pivot_table(
    df,
    values="total_sale",
    index="city",
```

```
    columns="category",  
    aggfunc="sum",  
    fill_value=0  
)
```

pivot

Generate Executive Report

```
report = pd.DataFrame(  
    "Metric":[  
        "Total Revenue",  
        "Average Sale",  
        "Max Sale",  
        "Min Sale",  
        "Number of Customers"  
    ],  
    "Value":[  
        df["total_sale"].sum(),  
        df["total_sale"].mean(),  
        df["total_sale"].max(),  
        df["total_sale"].min(),  
        df["customer_name"].nunique()  
    ]  
)
```

report

Export Results

CSV

```
df.to_csv(
    "cleaned_sales_data.csv",
    index=False
)
```

JSON

```
df.to_json(
    "sales_data.json",
    orient="records",
    indent=4
)
```

Excel

```
df.to_excel(
    "sales_data.xlsx",
    index=False
)
```

Summary Report

```
report.to_csv(
    "sales_report.csv",
    index=False
)
```

Advanced Challenges

Challenge 1: Incremental Load

Load only today's sales:

```
SELECT *  
FROM sales  
WHERE sale_date = CURRENT_DATE;
```

Challenge 2: Monthly Revenue Trend

Calculate monthly revenue using:

```
df.groupby("month_name")["total_sale"].sum()
```

Challenge 3: Data Quality Rules

Check:

- Null values
- Duplicate records
- Negative quantities
- Negative prices
- Invalid dates

Generate a data quality report.

Challenge 4: Load Back into PostgreSQL

Create a new table:

```
processed_sales
```

Then write the cleaned data:

```
df.to_sql(  
    "processed_sales",  
    engine,  
    if_exists="replace",
```

```
    index=False  
)
```